

Documentazione Rotor

SCL1054-V4

Tre funzioni: port expander (1 porta bidirezionale, 2 solo output + 1 singolo pin per selezione banco ROM), espansione RAM (192 byte extra) e logica decodifica indirizzi.

La RAM è mappata nel range di indirizzi 0x0144-0x1FF ed è accessibile come fosse RAM interna al micro

Le porte I/O sono controllate tramite 5 registri mappati in memoria (i nomi corrispondono a quelli usati nel disassembly del codice):

1. **SCL_DATA**: bidirezionale; registro "direzione" @ 0x0023 (bit a 1 → il pin è configurato come output), registro dati @ 0x0022. Porta a 8 bit, prende il nome di "SCL_DATA" perché, come visibile dagli schemi, funge da bus dati esterno per le varie periferiche collegate al SCL
2. **SCL_ADDR**: 8 bit, output only; registro dati @ 0x0020. Pilota, tramite U5 e U6 il bus indirizzi di ROM/RAM esterne. I bit 7, 6, 5, 3 sono multiplexati con i segnali di controllo dei tre solenoidi + il reset del LCD (vedi schema)
3. **SCL_CTRL**: 8 bit, output only; registro dati @ 0x0021. Collezione di vari output, controlla lo schema per più dettagli. (Nello schema, i pin di questa porta sono indicati come SCL-*nn*, dove *nn* è il numero del pin nel package del SCL; SCL-43 corrisponde al bit 0, e gli altri seguono in ordine crescente)
4. **SCL_BANK**: 1 bit, output only; Pilota A14 sulla ROM: permette quindi di scegliere fra i due banchi di codice.

Tutta la logica di decodifica degli indirizzi di memoria per le porte, la RAM e la ROM, è contenuta nell'SCL, che procede a selezionare automaticamente i circuiti corrispondenti.

Comunicazione dati

Il Rotor comunica con il "traslatore di centrale" (di seguito indicato semplicemente come *TRS*) utilizzando la linea as/bs, dalla quale viene anche prelevata la corrente necessaria per l'alimentazione del circuito caricabatterie.

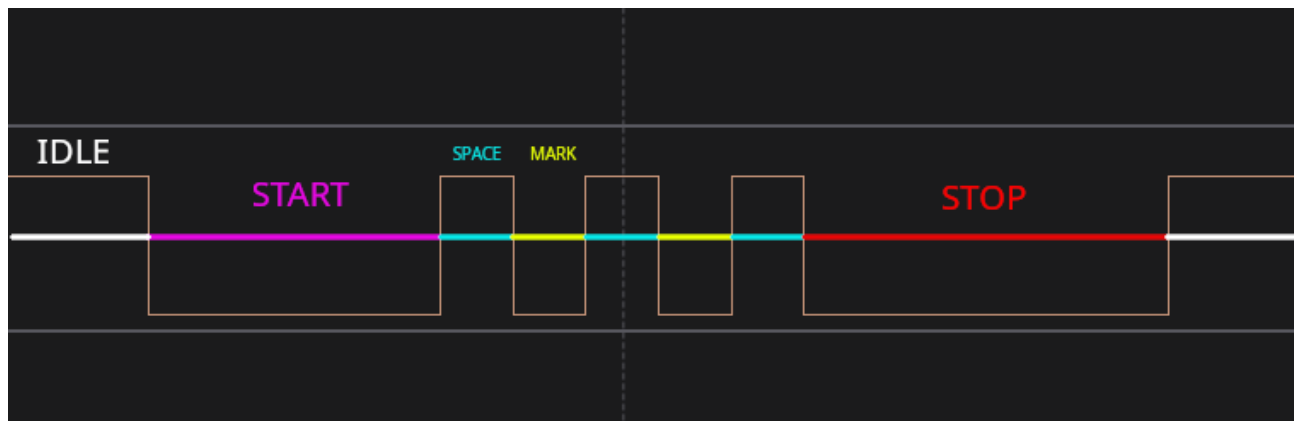
Le comunicazioni in direzione *TRS*→*Rotor* avvengono invertendo la polarità della linea di segnalazione (polarità normale: filo a NEGATIVO, filo b POSITIVO; **N.B.**: i sistemi telefonici sono a TERRA POSITIVA, quindi è il filo B ad essere a terra) in modo temporizzato; queste inversioni vengono rilevate dal telefono mediante il fotoaccoppiatore **OP3**.

Le comunicazioni in direzione *Rotor*→*TRS* avvengono invece variando la corrente assorbita dal telefono sulla linea di segnalazione: quando il telefono desidera trasmettere, il fotoaccoppiatore **OP4** viene attivato, cortocircuitando i tre transistor *T5, T6, T7* e aumentando la corrente di linea; il TRS rileva questa variazione mediante un proprio fotoaccoppiatore inserito in serie alla linea.

Nota: questo tipo di comunicazione è stato "ereditato" dal precedente telefono modello *G+M*, tant'è che nel brevetto di uno dei TRS compatibili con il Rotor il circuito di interfaccia è indicato con il nome di SGM ("Sonda Gettone+Moneta")

Esistono due diversi "protocolli" di comunicazione, utilizzati in situazioni differenti:

1) Protocollo "Telegrafico":



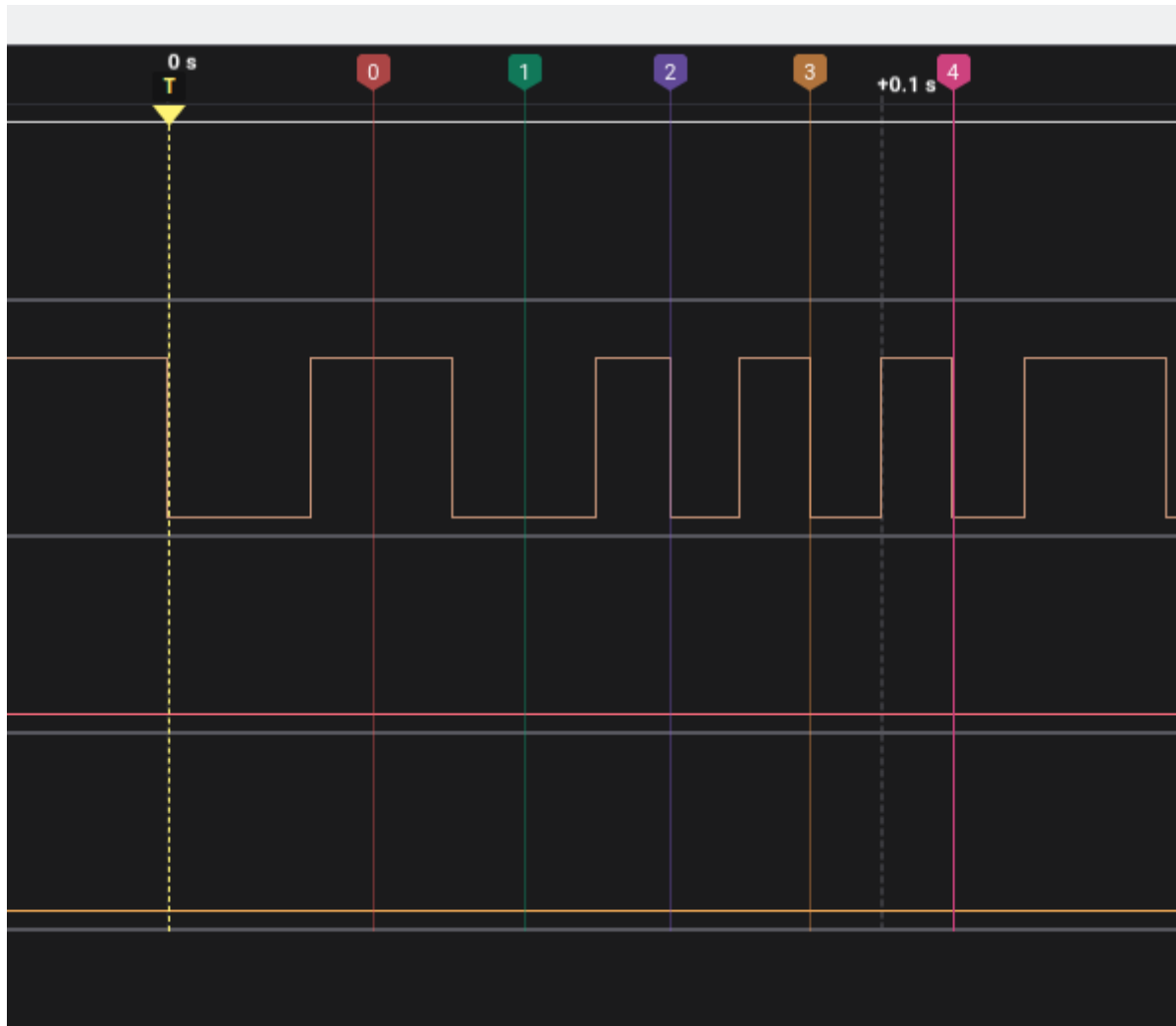
Si tratta del più semplice dei due, ed è simile alla selezione decadica; una comunicazione di questo tipo si può analizzare dividendola in 5 fasi, come visibile nell'immagine.

N.B: I segnali sono mostrati nella forma in cui appaiono sulle porte di I/O del microcontrollore. I segnali in entrambe le direzioni (Rotor→TRS e TRS→Rotor) assumono in questo caso lo stesso aspetto in termini di forme d'onda.

Per i segnali provenienti dal TRS, il livello logico ALTO corrisponde a polarità *normale*, mentre quello basso a polarità *invertita*; per i segnali provenienti dal Rotor, il livello ALTO corrisponde a OP4 *off* e quindi corrente di linea minima (la sola corrente di alimentazione), mentre quello basso corrisponde a OP4 *on* e corrente massima

- A) **IDLE:** linea "a riposo"
- B) **START:** impulso di inizio comunicazione. Per le trasmissioni provenienti dal Rotor, ha durata fissa a 40 ms; Per le trasmissioni provenienti dal TRS, può avere durata 40ms OPPURE altri tre valori, più lunghi (le durate esatte sono ancora da determinare). Due dei tre possibili valori (i due più corti nello specifico) per la durata dell'impulso fanno sì che i dati ricevuti vengano interpretati in modo differente (possibilmente come dati/comandi/ecc, i dettagli sono ancora da determinare); il terzo ed il quarto sono, in ordine di lunghezza, rispettivamente un comando di reset per il telefono ed un comando di reset per variabili legate al LIOD.
- C) **SPACE, MARK:** In seguito all'impulso di start, i dati da trasmettere sono inviati cifra (decimale) per cifra sotto forma di una serie di impulsi negativi (*mark*), lunghi 10ms, separati da pause della stessa lunghezza (*space*). La cifra trasmessa è pari al numero di mark + 1; lo 0 è trasmesso sotto forma di 10 impulsi.
- D) **STOP:** impulso di fine trasmissione. Ha sempre durata 50ms in direzione Rotor→TRS, mentre in direzione opposta sono ammesse due differenti lunghezze che, di nuovo, cambiano il modo in cui vengono interpretati i dati trasmessi.

2) Protocollo "pseudo-BPSK"



Si tratta del secondo protocollo supportato, utilizzato principalmente per inviare i dati di carte di credito (telefoniche e/o "commerciali" di tipo Diners e AMEX) e schede telefoniche prepagate.

In questo caso la trasmissione è di tipo binario e, da quanto determinato finora, sembra sia utilizzata solo in direzione Rotor → TRS

La trasmissione con questo protocollo inizia sempre con uno start, composto da un picco negativo lungo 20ms seguito da 10ms di pausa (nella foto, è la parte fra il cursore *T* ed il cursore *O*), seguito poi dai dati binari.

Tali dati sono codificati con quella che possiamo definire codifica pseudo-BPSK (*Binary Phase Shift Keying*):

- Un bit a **0** è codificato come 10ms a livello alto seguiti da 10ms a livello basso
- Un bit a **1** è codificato come 10ms a livello basso seguiti da 10ms a livello alto

Nella foto di esempio, i primi 4 bit trasmessi sono **0111** (→ 0x7)

Comunicazione lettore-Rotor:

Come molti aspetti di questo telefono, la comunicazione fra lettore di schede interno (d'ora in poi indicato come *LIOD*, "Lettore Integrato a Obliterazione Durante", come viene chiamato nei manuali) avviene in modo assolutamente non standard.

A livello elettrico, la comunicazione avviene tramite due linee unidirezionali; d'ora in poi indicheremo come *TXD* la linea che sposta dati in direzione Rotor→LIOD e *RXD* quella in direzione opposta (N.B: nello schema le linee sono indicate dal punto di vista del LIOD, quindi invertite rispetto a questo documento).

Un'ulteriore linea viene utilizzata dal telefono per rilevare la presenza del lettore; un resistore di pull'up all'interno del lettore fa sì che questa linea sia a +5V con lettore presente, 0 altrimenti.

Nota: La scheda di interfaccia lettore (indicata d'ora in poi come *MIA*, "Modulo Interfaccia Aggiuntivi", di nuovo per coerenza con il manuale) è decisamente più complicata di quanto necessario in quanto è concepita per supportare due differenti tipi di lettore.

Non sappiamo di per certo quale sia il secondo lettore teoricamente supportato (la documentazione lo indica semplicemente come *lettore di debito*, ma il manuale, almeno la versione più recente, indica che l'unico lettore effettivamente utilizzato è il LIOD); un'ipotesi è che si tratti del mitico "lettore azzurro", un lettore di schede telefoniche progettato per l'uso con il *G+M* e prodotto in numeri molto ridotti, ma non dispongo di un lettore per confermare questa ipotesi e non sappiamo nemmeno se il codice lato Rotor sia completo e funzionante.

Un ulteriore pin del microcontrollore viene utilizzato per scegliere fra i due e l'interfaccia per l'altro lettore utilizza dei pin in più, ma per semplicità ignoreremo completamente questa possibilità.

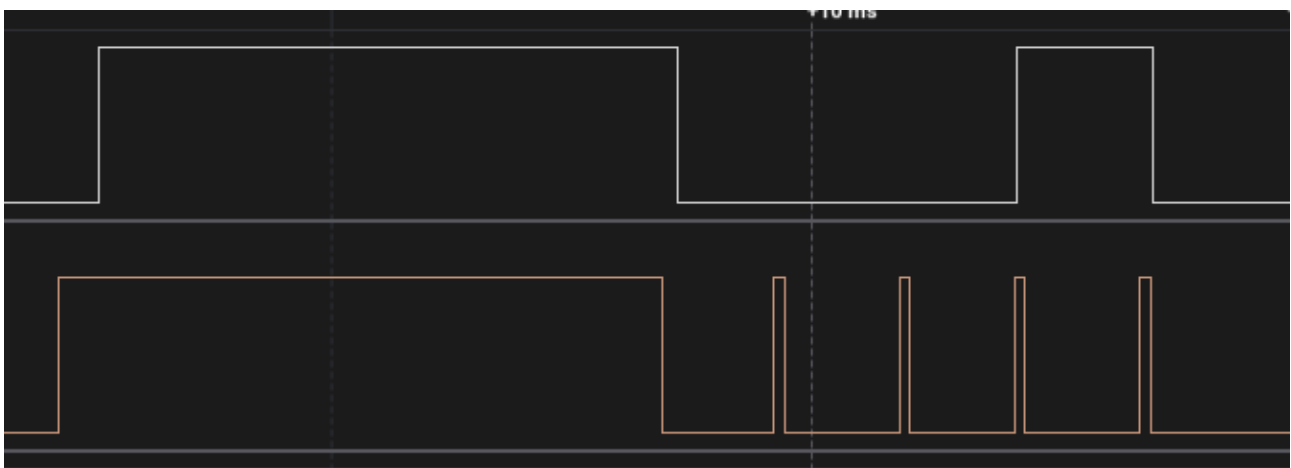
Tutte le foto e le descrizioni da qui in poi fanno riferimento ai segnali come visti dal microcontrollore del Rotor. In ogni caso, una volta che il telefono è inizializzato ed ha rilevato correttamente il lettore, il MIA passa il segnale *RXD* (e DET) inalterato dal lettore al telefono, mentre *TXD* viene invertito. Il successivo Rotor O.V. fa a meno del MIA e controlla il lettore direttamente dal micro.

Nota 2: Esistono diverse revisioni del LIOD, l'ultima delle quali viene indicata nel manuale come LI O.D./R.I ("Lettore Integrato a Obliterazione Durante e Restituzione dalla bocchetta di Ingresso). Il manuale fornisce ulteriori dettagli a tale proposito; l'interfaccia rimane la medesima.

Il protocollo di comunicazione non è né *RS232* standard, né tanto meno una delle interfacce sincrone "comuni" quali SPI, I2C ecc.

Il Rotor agisce sempre come *master* della comunicazione, in quanto il lettore non può iniziare una comunicazione ma solo inviare dati quando clockato dal telefono.

Vediamo un esempio di una comunicazione:



In bianco *RXD*, in arancio *TXD*; *TXD*, come intuibile dall'immagine, ha la doppia funzione di trasportare comandi in direzione Rotor→LIOD e di clockare il lettore per la risposta.

La comunicazione si articola in due fasi:

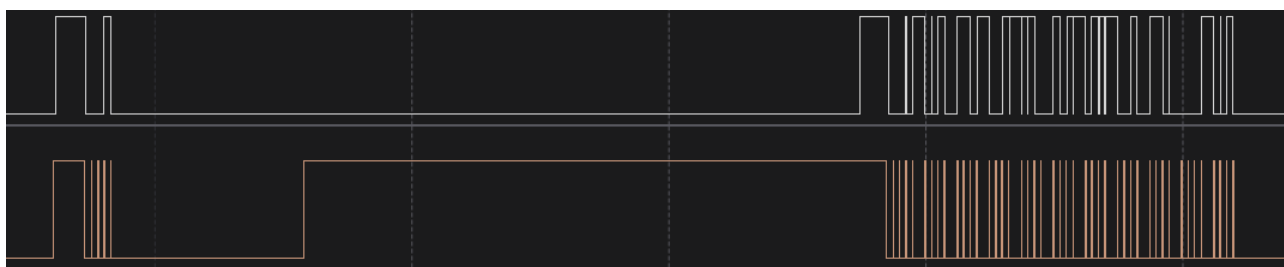
1. **Comando:** (sempre proveniente dal telefono e destinato al lettore) si tratta del primo impulso positivo; i differenti comandi sono codificati variando la lunghezza di tale impulso. Il firmware

del Rotor 2 versione 1.4, l'ultima rilasciata, supporta 10 comandi; il lettore è in grado tuttavia di riconoscerne un numero più ampio (potrebbero essere comandi di test/comandi avanzati supportati solo dal Rotor OV). Il lettore risponde semplicemente pilotando *RXD* allo stesso livello di *TXD* (salvo casi particolari indicati più avanti)

2. **Clock + risposta:** Il telefono a questo punto procede a clockare il lettore in modo da leggere la risposta inviata da quest'ultimo. Ciascuno dei quattro impulsi di clock inviati dal telefono ha **entrambi i fronti attivi**: *RXD* viene campionato quindi due volte per impulso, una sul fronte di salita ed una sul fronte di discesa, con un piccolo delay fra il fronte di clock ed il campionamento per consentire al lettore di pilotare *RXD* al valore appropriato.

Nella trasmissione di esempio, il telefono invia il comando "di idle" (comando che viene inviato continuamente dal telefono per segnalare al lettore che è tutto ok e può accettare schede), mentre il lettore risponde con **0x07**, che sembra essere la risposta corrispondente a "L10D OK"

Esiste, come accennato sopra, un caso particolare: nel caso in cui il lettore voglia segnalare al telefono che è stata inserita una scheda (possibilmente esistono altri tipi di segnalazioni; il codice non è ancora stato analizzato a fondo), si comporta come visibile al centro della foto seguente:



Il lettore, cioè, rilevato l'inizio del comando, non risponde portando *RXD* a livello alto (quasi) istantaneamente, come di consueto, ma lo fa dopo un lungo delay; il telefono, rilevando la situazione "anomala", riconosce che una scheda è stata inserita e, una volta terminato un impulso di comando "non valido", procede ad inviare una lunga sequenza di clock per estrarre tutti i byte del contenuto della scheda.

Sequenza di boot del telefono:

Il telefono, per funzionare correttamente, richiede due cose:

1. Batteria tampone principale (6V piombo) sufficientemente carica.
2. Configurazione esterna e contatori gestionali (contenuti nella RAM sulla board "RAM esterna") validi

La batteria tampone della RAM esterna, che consente al telefono di mantenere configurazione esterna e contatori gestionali memorizzati anche con batteria tampone principale scarica, non è strettamente necessaria per il funzionamento del telefono.

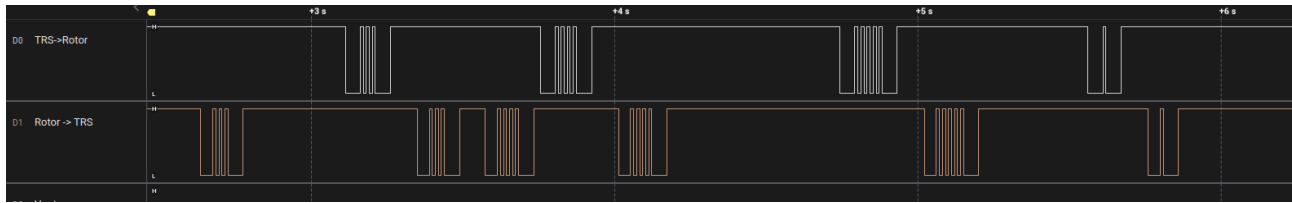
Se la condizione numero 2 è verificata, il telefono può essere avviato in modalità normale (entrambi i dipswitch accanto al display su off) semplicemente premendo reset; terminato il boot, il telefono visualizza **F.S. (con LED di "fuori servizio" spento)** e si pone in attesa della configurazione remota dal TRS

Altrimenti, se i dati di configurazione non sono validi (ad es. telefono con batterie scariche appena rimesso in servizio), è necessario avviare il telefono con il microswitch di configurazione (il sx, vedi manuale) in posizione ON; questo riscriverà la configurazione in RAM.

Se, inoltre, i contatori gestionali (totale monete in cassaforte ecc) risultano non validi, è necessario avviare il telefono tenendo premuto (**dal reset fino all'avvio completo**) il *pulsante del gestore*, che è il pulsantino posto all'interno dello scomparto della cassaforte, visibile con cassetta rimossa.

Sequenza di configurazione remota:

Il telefono, dopo ogni boot avvenuto con successo, si pone in *F.S.* in attesa della configurazione remota; la procedura in questione è piuttosto semplice, ed è visibile nell'immagine seguente



(riferita ad un telefono con LIOD installato e funzionante).

La procedura prevede la seguente sequenza (tutta eseguita usando il metodo di configurazione telegrafico):

1. Il telefono invia un **3** (prima cifra dello "stato lettore" visualizzato sul display all'avvio, cioè "30", "33" o "34", vedi manuale)
2. Il TRS risponde con un **3**
3. Il telefono ripete il **3** precedente, quindi invia subito la seconda cifra dello stato lettore (in questo caso **4**, lettore OK); Se il lettore è assente ("30" visualizzato sul display all'avvio), questo passaggio ed i due successivi vengono ignorati
4. Il TRS risponde ripetendo l'ultima cifra ricevuta e solo l'ultima (qui un **4**)
5. Il telefono ripete la cifra appena ricevuta dal TRS (di nuovo **4**)
6. Il TRS ora invia **5** (valore fisso hardcoded)
7. Il telefono risponde **5**
8. Il TRS invia **1** (di nuovo un valore fisso)
9. Il telefono risponde **1**
10. Il TRS invia quindi, una per una, le tre cifre corrispondenti al valore in lire del gettone telefonico (lo 0 è inviato con 10 impulsi); dopo ogni cifra il telefono risponde reinviano la cifra appena ricevuta al TRS.
11. La procedura di configurazione è ora conclusa ed il telefono è ora funzionante

Nota 1 sul punto 10: la prima cifra del costo del gettone non può essere 0

Nota 2 sul punto 10: a causa di un bug del codice, anche se il telefono può teoricamente accettare qualsiasi valore per le ultime due cifre del costo gettone (es: 123 lire, 488 lire ecc), solo valori multipli di 100 sono effettivamente validi (nello specifico, le ultime due cifre sono salvate in memoria, ma in diversi punti il codice è scritto partendo dal presupposto che il costo sia multiplo di 100 lire e quindi tali cifre sono ignorate)

Una volta terminata la configurazione iniziale, il telefono risulta funzionante e non necessita di ulteriori comunicazioni col TRS per essere utilizzato come un normale telefono. Si comporta sostanzialmente come un Rotor 1: non incassa monete, ma permette di effettuare e ricevere telefonate. Tuttavia, il lettore di schede cessa di funzionare dopo un certo periodo di tempo dal boot (il telefono smette di inviare comandi di idle al lettore, che cessa di accettare schede pur non bloccandone l'inserimento tramite l'apposito solenoide nella bocchetta). Probabilmente è necessario qualche sorta di comando di "keepalive" da parte del TRS, ma tale comando è sconosciuto

Comandi TRS:

Al momento, non si conoscono sufficienti dettagli sui comandi o sul loro significato; questa sezione verrà quindi espansa.

Informazioni note al momento:

- **3** causa l'espulsione della scheda inserita nel lettore, se presente
- **5** e **7** provocano una risposta dal telefono, ma il significato di tale risposta dal telefono

Note varie:

Importante: se il lettore legge ed accetta una scheda prepagata (quindi una scheda leggibile e con credito $\neq 0$), questa viene trattenuta dal telefono in attesa di risposta dal TRS. **Per riottenere indietro la scheda NON resettare il telefono:** questo ne causa la cancellazione!! (come misura di sicurezza contro truffe con le schede). Il metodo corretto è di inviare un **3** da un TRS simulato, oppure disconnettere la linea telefonica di segnalazione, ed aspettare poi che il telefono invii al LIOD il comando di espulsione scheda.